	PROCESO: DESARROLLO DE SOFTWARE / PERSISTENCIA E INFRAESTRUCTURA	**ELABORACIÓN:** 11/09/2026
	IMPLEMENTACIÓN DE TABLAS, ÍNDICES Y SCRIPTS DDL EN POSTGRESQL 18	**ÚLTIMA REVISIÓN:** 11/09/2026

Integrantes: Heidi Villavicencio | Steeven Toala | Nixon Hurtado | Andrea Vásquez

1. TÍTULO

Denominación del Entregable: Implementación de Tablas, Índices y Scripts DDL en PostgreSQL 18 (Node.js / Express v5) (EDT 3.1)

2. INTRODUCCIÓN

En la transición del diseño relacional a la fase de construcción del sistema **SIGE**, la capa de persistencia requiere materializar el modelo lógico en el motor relacional **PostgreSQL (18)**. Se implementa una arquitectura basada en scripts DDL deterministas (`schema.sql`), ejecución automatizada mediante el script de arranque `init.js` en Node.js y gestión de transacciones de alto rendimiento mediante el connection pool de la librería oficial `pg` (`backend/src/config/db.js`).

Este documento detalla la implementación técnica del esquema físico en PostgreSQL 18, la creación de las tablas relacionales con soporte UUID v4 nativo (`gen_random_uuid()`), restricciones de integridad `CHECK`, índices compuestos optimizados y pruebas de concurrencia ACID para el ciclo de vida documental del Sistema de Gestión de Calidad (SGC).

3. OBJETIVO

3.1. Objetivo General

Implementar el modelo relacional físico en PostgreSQL 18 mediante scripts DDL deterministas y el pool de conexiones en Node.js + Express (v5), estableciendo esquemas normalizados en 3FN, índices de rendimiento y restricciones de integridad para el ciclo de vida documental del SGC.

3.2. Objetivos Específicos

- Diseñar y parametrizar el script DDL `schema.sql` para la creación de las tablas relacionales con tipos UUID, timestamps UTC e integridad referencial.
- Configurar el módulo `backend/src/config/db.js` con connection pool `pg.Pool` optimizado para peticiones concurrentes.
- Desarrollar el script de inicialización `backend/src/database/init.js` para la creación desatendida del esquema en entornos locales y contenedores.
- Implementar índices B-Tree únicos y compuestos para la auditoría inmutable, control de plazos extemporáneos y circuito secuencial de firmas en cascada.

4. ALCANCE

4.1. Alcance Incluido

Mapeo de entidades: `carreras_unidades`, `usuarios`, `roles`, `auth_tokens`, `catalogo_actividades`, `documentos`, `secciones_documento`, `habilitaciones_extemporaneas`, `solicitudes_modificacion_plan`, `actividades_matriz`, `evidencias_minio`, `firmas_legalizacion`, `observaciones_comite`, `historial_cambios` y `auditoria_logs`. Restricciones `CHECK` para ponderaciones y orden de firmas (1, 2, 3).

4.2. Límites y Exclusiones

No incluye el servicio de almacenamiento de blobs binarios (delegado a MinIO en EDT 3.11) ni la capa de consultas OLAP.

5. DEFINICIONES

- **PostgreSQL 18:** Motor de base de datos relacional de código abierto que ofrece transacciones ACID y alto rendimiento.
- **Connection Pool (`pg.Pool`):** Mecanismo de reuso de conexiones TCP abiertas hacia PostgreSQL para reducir la latencia de handshake.
- **Script DDL (*Data Definition Language*):** Conjunto de sentencias SQL que estructuran el almacenamiento relacional.
- **EDT (*Estructura de Desglose del Trabajo*):** Descomposición jerárquica del proyecto orientada a entregables verificables.

6. RESPONSABILIDADES

Integrante	Rol Asignado	Responsabilidad en este Entregable
Nixon Hurtado	Backend / DBA	Escritura de scripts DDL `schema.sql`, configuración del pool pg y optimización de índices.
Steeven Toala	Líder / Arquitecto	Diseño de la arquitectura de inicialización `init.js`, revisión de restricciones y Docker compose.
Ing. Paulo Torres	Docente Tutor	Supervisión técnica y aprobación de la consistencia del esquema físico en PostgreSQL 18.

Cuadro 1: Matriz de Responsabilidades del Entregable EDT 3.1.

7. NORMATIVA LEGAL Y TÉCNICA

- **ANSI/ISO/IEC 9075:2023:** Estándar de Lenguaje SQL.
- **ISO/IEC 25010:** Atributos de fiabilidad, tolerancia a fallos e integridad de datos.
- **Reglamento del SGC - FISEI UTA:** Obligatoriedad de conservación íntegra de evidencias y foliación de expedientes.

8. MÉTODO, POLÍTICAS Y CONTENIDOS

8.1. Metodología

Construcción directa y modular de scripts SQL parametrizados ejecutados mediante el driver nativo `pg` en Node.js, integrados en el ciclo de vida del contenedor Docker `sige-postgres`.

8.2. Políticas de Persistencia

- Toda tabla contiene clave primaria UUID inmutable y columnas de trazabilidad temporal (`created_at` con `DEFAULT CURRENT_TIMESTAMP`).
- Se prohíbe el borrado físico (`CASCADE`) en tablas transaccionales de auditoría, firmas y documentos legalizados.

9. DESCRIPCIÓN DE ACTIVIDADES

9.1. Estructura de Persistencia en `backend/src/database/schema.sql`

A continuación se ilustra la definición SQL de la tabla de firmas en cascada y sus índices únicos:

```sql
CREATE EXTENSION IF NOT EXISTS "pgcrypto";

CREATE TABLE IF NOT EXISTS firmas_legalizacion (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    documento_id UUID NOT NULL REFERENCES documentos(id) ON DELETE RESTRICT,
    usuario_firmante_id UUID NOT NULL REFERENCES usuarios(id) ON DELETE RESTRICT,
    fase_documento VARCHAR(30) NOT NULL CHECK (fase_documento IN ('PLANIFICACION_INICIAL', 'INFORME_FINAL')),
    orden_cascada INT NOT NULL CHECK (orden_cascada IN (1, 2, 3)),
    rol_instancia VARCHAR(40) NOT NULL CHECK (rol_instancia IN ('DOCENTE_RESPONSABLE', 'COORD_AREA_CONOCIMIENTO', 'COORD_CARRERA')),
    estado_firma VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE' CHECK (estado_firma IN ('PENDIENTE', 'FIRMADO', 'RECHAZADO')),
    serial_certificado VARCHAR(150),
    fecha_firma TIMESTAMP WITH TIME ZONE,
    sello_digital_base64 TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Restriccion de unicidad estricta para la cascada jerarquica
CREATE UNIQUE INDEX IF NOT EXISTS uq_firmas_documento_fase_orden
ON firmas_legalizacion(documento_id, fase_documento, orden_cascada);
```

9.2. Script de Inicialización Automatizada (`init.js`)

```javascript
// backend/src/database/init.js
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import pool from '../config/db.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

export const initializeDatabase = async () => {
  try {
    const schemaPath = path.join(__dirname, 'schema.sql');
    const schemaSql = fs.readFileSync(schemaPath, 'utf-8');
    await pool.query(schemaSql);
    console.log('PostgreSQL 18: Esquema DDL inicializado exitosamente.');
  } catch (error) {
    console.error('Error al inicializar esquema PostgreSQL 18:', error.message);
    throw error;
  }
};
```

9.3. Verificación de Integridad y Concurrencia

Se ejecutó una batería de inserciones concurrentes validando el bloqueo de duplicados en el circuito de firmas, el cumplimiento del rango de ponderaciones ($0.00 \leq \text{ponderacion} \leq 100.00$) y la persistencia de timestamps automáticos sin sobrecarga del motor.

10. FIRMAS DE RESPONSABILIDAD

ACCIÓN	NOMBRE Y CARGO	FIRMA	FECHA
Elaborado por:	Heidi Villavicencio Analista de Requisitos / UI-UX		11/09/2026
Revisado por:	Steeven Toala Líder de Proyecto / Arquitecto		11/09/2026
Revisado por:	Nixon Hurtado Desarrollador Backend / DBA	**[FIRMADO ELECTRÓNICAMENTE]**	11/09/2026
Revisado por:	Andrea Vásquez Desarrolladora Frontend / QA	**[FIRMADO ELECTRÓNICAMENTE]**	11/09/2026
Aprobado por:	Ing. Paulo Torres, Mg. Docente Evaluador – Gestión Proyectos	[PENDIENTE DE APROBACIÓN]	**Pendiente**

11. CONTROL DE CAMBIOS

Versión	Descripción del cambio	Responsable del cambio	Fecha de aprobación
1.0	Implementación de esquema DDL en PostgreSQL 18, índices y script de inicialización init.js	Nixon Hurtado	10/09/2026

12. ANEXOS

Anexo 1: Configuración del Pool de Conexiones en `backend/src/config/db.js`

Archivo de conexión modular que utiliza la librería `pg` con variables de entorno (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`), tamaño de pool máximo de 20 conexiones y manejo de eventos de error de red.